The EYES Manual

Zifei Zhong
(draft 2)

October 2, 2026

Contents

1 Design Purpose **3**

2 Data Characteristics of the Chronicle Log Files **3**

2.1 Data File Format 3

3 File Categorization **3**

3.1 Levels of the Decision Tree 4

3.2 Processing Groups 5

3.3 Attributes Reported with Each File 6

4 Design Philosophy **6**

5 Prerequisites **7**

6 Launching the Toolbox **7**

7 Execution Flow **7**

7.1 Phase A: Core Pre-processing 7

7.2 Phase B: Segment Analysis 9

7.2.1 Glance Logic 10
7.3 Phase C: Application & Event Logic 12
7.4 Sessions 14
7.5 Phase D: Final Outputs 16

8 Usage Commands **16**

9 Log Events defined in the Android System **17**

9.1 Event Classes 17
9.2 Shutdown, Startup and User Unlocked 20

10 Device State Analysis **21**

10.1 Framework 22
10.2 Model 22
10.3 Step 1: Off or On 23
10.4 Boot signature 25
10.5 Step 1 results (A1) 27
10.6 Step 2: Idle or Active 27
10.7 Reviewing quiet stretches 30
10.8 Results (A1) 30

1 Design Purpose

The EYES toolbox is designed to process and analyze log files collected from Android smart devices (phones and tablets) by the App Chronicle. The toolbox generates outputs such as App Usage, Device Pickups, Visualization, and integrated event timelines. The toolbox can be launched either from the command line or from a desktop GUI, where a user selects a folder containing raw Chronicle `.csv` files and runs the full pipeline with one click.

2 Data Characteristics of the Chronicle Log Files

The Chronicle log files are collected on a user device and then uploaded to a server. Some characteristics that we have observed in the data include:

1. Missing data: Some event logs might not be uploaded to the server successfully, thus causing missing data. For example, there may be a 'Screen Interactive' event in the log file, but the corresponding 'Screen Non-interactive' event is missing.
2. Duplication: Some event logs are uploaded multiple times, thus causing duplicated rows in the same file.
3. Out-of-order rows: In general, the log file records rows in increasing time order based on 'event_timestamp'. However, this is not always true. Many files contain rows recorded out of order.

There might be other issues that we are not aware of. We will keep the document updated when we have new observations.

2.1 Data File Format

The EYES toolbox takes raw data files as input. The raw data files are '.csv' files in which each row logs an event, including information such as 'app_package_name', 'interaction_type', and 'event_timestamp'. Figure 1 shows the raw data file format.

One recommended practice is to put all raw files in one directory and provide the path to this directory to the EYES toolbox. Then all raw data files in that directory will be processed.

3 File Categorization

Chronicle log files differ in ways that matter for analysis and for processing. Before any processing, each raw file is assigned to exactly one *file group* by the decision tree in Figure 2. The tree uses

4075_C01_AndroidTab Chronicle Android Raw Data 02-28-2025

study_id	participant_id	app_package_name	interaction_type	event_type	event_timestamp	timezone	username	application_label
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Configuration Change		2024-09-23T12:25:30.108-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Configuration Change		2024-09-23T12:25:51.807-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Configuration Change		2024-09-23T12:25:52.945-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.samsung.android.lool	Unknown importance: 10		2024-09-23T12:25:57.414-04:00	America/New_York		Device care
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.soundalive	Unknown importance: 11		2024-09-23T12:25:57.947-04:00	America/New_York		SoundAlive
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.samsung.tvmode	Unknown importance: 11		2024-09-23T12:25:58.099-04:00	America/New_York		TV Mode
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Configuration Change		2024-09-23T12:25:59.954-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Move to Background		2024-09-23T12:26:00.024-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Move to Foreground		2024-09-23T12:26:00.042-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Unknown importance: 23		2024-09-23T12:26:00.572-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Unknown importance: 16		2024-09-23T12:26:54.525-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Unknown importance: 15		2024-09-23T12:26:54.663-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Unknown importance: 17		2024-09-23T12:26:54.706-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Move to Background		2024-09-23T12:26:54.956-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Unknown importance: 23		2024-09-23T12:26:55.008-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Move to Foreground		2024-09-23T12:26:56.791-04:00	America/New_York		Camera

Figure 1: Raw data file format.

only properties that can be determined from the file itself, so every new file can be classified the same way.

3.1 Levels of the Decision Tree

1. **Role.** Parent or child, from the participant ID in the file name (`P0x` or `C0x`). Parents' and children's data are kept apart throughout, because they answer different research questions. Role does not change how a file is processed.

2. **Device type.** Read from the device label in the file name: `Fire` is a Fire tablet; `Android`, `AndroidPn` and `AndroidTab` (including misspellings such as `Andriod`) are Android phones or tablets. A file is a Chromebook, whatever its label, if packages starting with `org.chromium.arc` make up at least 1% of its events (Android apps on ChromeOS run through ARC). A label that is not recognized is treated as an Android phone or tablet and flagged for review.

3. **Logs *Activity Stopped*.** Yes if the file contains at least one *Activity Stopped* event (named `Activity Stopped` in new-format files, `Unknown importance: 23` in old-format files). This depends on the Android version: *Activity Stopped* exists from Android 10, so devices on Android 9 or older (for Fire tablets, Fire OS 7) never log it. In our data this property never changed between exports of the same device.

Levels 2 and 3 determine the *processing group* (A1, A0, F1, F0, CB), which decides how a file is processed. Together with the role, they give the file group, written *role-processing group*: for example **P-A1** is a parent's Android device that logs *Activity Stopped*, and **C-F0** is a child's Fire tablet that does not. Files in **P-A1** and **C-A1** are processed identically but are kept in separate groups.

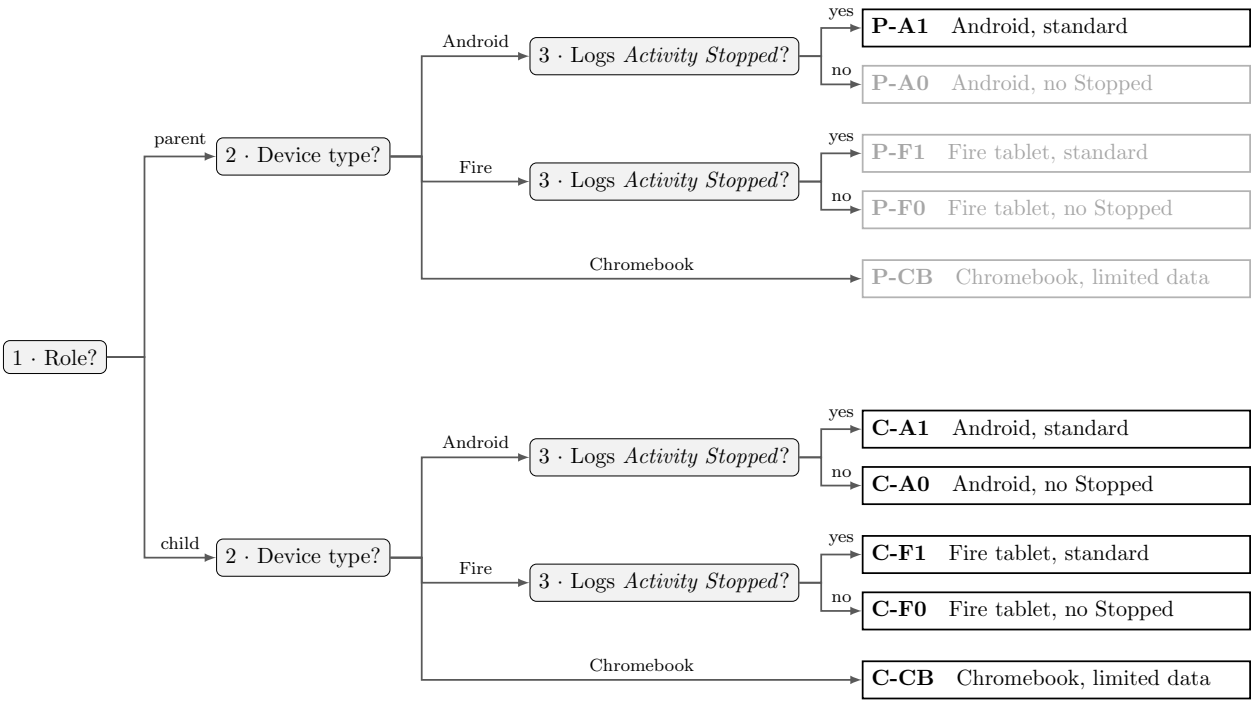


Figure 2: Decision tree for assigning a raw file to a file group. Levels 2 and 3 decide how the file is processed; level 1 keeps parents' and children's data apart. Grey leaves have no files in the current data but complete the tree for future files.

3.2 Processing Groups

Group	Rule	Processing
A1	Android phone or tablet that logs *Activity Stopped*	Standard.
A0	Android phone or tablet without *Activity Stopped*	App usage ends at *Activity Paused*; trigger chains are built from timing only; picture-in-picture use is taken from the app catalog, because it cannot be detected in the log.
F1	Fire tablet that logs *Activity Stopped*	Standard. Unlocks are often not logged (Amazon Kids profile), so GLANCE relies on the foreground-app evidence.
F0	Fire tablet without *Activity Stopped*	As A0.
CB	Chromebook	Standard rules, but only Android apps are logged: Chrome browser use and some screen wakes are missing, so usage is undercounted.

A Chromebook is a single processing group whether or not it logs *Activity Stopped*, because its missing browser data limits the results either way.

3.3 Attributes Reported with Each File

The following properties are recorded for every file and reported with its results, but they are not levels of the tree:

- **Log format**: old (*Move to Foreground / Background*, system events as codes), new (*Activity Resumed / Paused / Stopped*, system events by name), or switching from old to new within the file. The toolbox treats both sets of names as the same events.
- **Unlock group**: *Keyguard Hidden* events per *Screen Interactive* event: A (none), B (under 2%), C (2–20%), D (20% or more). It indicates how much GLANCE detection relies on foreground-app evidence rather than logged unlocks.

Keyguard Hidden is deliberately not a level of the tree. It does not change processing, because the GLANCE rule accepts either an unlock or foreground-app use as evidence of use (Section 7.2.1). It is also a user setting rather than a device capability: it varies continuously between devices, so any cut-off would be arbitrary, and it can change during a recording when a lock screen is added or removed.

4 Design Philosophy

The EYES toolbox analyzes the log data by:

- Identifying the internal states of the device: ACTIVE, GLANCE, IDLE, SHUTDOWN, GAP.
- Constructing the logic structure of an activity (triplet): Activity Resumed, Activity Paused, Activity Stopped.
- Inference: A set of inference rules is programmed into the toolbox scripts to ensure detailed analysis. For example, if no 'Activity Stopped' event exists, then the toolbox uses the pair ('Activity Resumed', 'Activity Paused') as the app usage interval; if only 'Activity Resumed' exists, then it uses the next app's 'Activity Resumed' event as the termination of the previous app usage interval.
- Combining the above to accurately extract App usage or device pickups.

Other data analysis tools also exist. For example, the Uzair (cite) scripts are widely used. However, the data processing logic of the Uzair tool is very different from that of the EYES toolbox. The Uzair tool simply matches each 'Activity Resumed' event to the next available 'Activity Paused' event and uses that interval as app usage. While this works for common app activity scenarios, it does not differentiate the activities of the same app well and it is not able to handle multi-window applications such as Youtube.

5 Prerequisites

The toolbox is developed in Python. Before running the EYES processing pipeline, ensure that you have:

1. Python 3 installed.
2. The EYES toolbox downloaded.
3. A terminal to run the execution commands (Unix/Linux terminal, or git bash on Windows), or access to the desktop GUI launcher.
4. A folder of raw Chronicle data files ready for processing.

6 Launching the Toolbox

The toolbox can be launched in two ways:

1. **GUI mode.** The GUI launcher allows a user to click `Browse`, select a folder containing raw Chronicle `.csv` files, and click `Go`. The GUI shows the current phase, elapsed run time, and command output in real time. This is the recommended mode for routine use by researchers who do not want to run multiple terminal commands manually.
2. **Command-line mode.** The toolbox can also be executed from a terminal by calling the `make` targets in the `Makefile`. This mode is convenient for debugging, scripting, or running only part of the pipeline.

7 Execution Flow

The pipeline follows a specific dependency chain. Each unit relies on the completion of the previous one. Figure 3 shows the dependency of the EYES processing units.

7.1 Phase A: Core Pre-processing

There are three units in the pre-processing phase: `preprocess, uniq, reorder`.

1. `preprocess`: This unit does filtering and sorting.
    - In general, raw Chronicle files do not need to be filtered. However, if a user wants to filter out certain rows based on 'interaction type', the scripts provide this functionality.

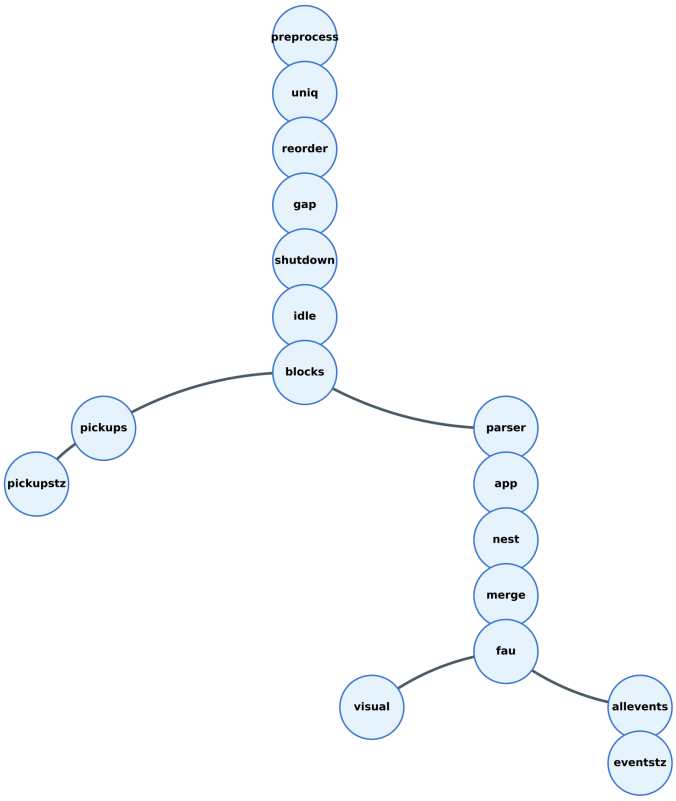


Figure 3: Dependency of the EYES Processing Units

For example, raw files often contain thousands of rows with 'Unknown importance: 11'. Filtering out these rows can reduce data size and speed up processing, but these rows are heartbeats (Section 9.1): they are the only record that the device was on when key events are missing, so they should be kept for manual review.

- Sorting is necessary. Since rows in the log file are not always recorded in time-increasing order, we sort the rows by 'event timestamp' to maintain a time-increasing order.

2. `uniq`: Removes duplicate entries.

   Once the data rows are sorted in the previous unit, `preprocess`, this unit, `uniq`, removes duplicated rows from the data file.

3. `reorder`: Reorder two particular events (Activity Resumed, Activity Paused) when they have identical timestamps.

   This is needed because identical timestamps create ambiguity: according to the timestamp, the two events happen at the same time, but logically one must occur before the other. The `reorder` unit is designed to place the 'Activity Paused' row before the 'Activity Resumed' row if both rows have identical timestamps. This resolves the ambiguity. If 'Activity Paused' is in fact behind 'Activity Resumed' (and thus creates a 0-duration interval between the two events), our `parser` unit is designed to catch it even if the rows are presented in the wrong order.

   For log files collected from old versions of the Android system, we match the tag 'Move to

Foreground' and 'Move to Background' to 'Activity Resumed' and 'Activity Paused', respectively.

7.2 Phase B: Segment Analysis

The processing units in this phase identify explicit non-active device states and then derive ACTIVE as the remaining time. The states are: ACTIVE, GLANCE, GAP, SHUTDOWN, and IDLE. The SHUTDOWN state indicates that a device is powered off. The IDLE state indicates that a device is in an idle mode in which the screen is off or the device is otherwise not interactive. The GAP state indicates that a device is in either SHUTDOWN or IDLE, but the toolbox cannot determine which state is more likely based on the available data. The GLANCE state indicates that the device screen briefly lights up but the device is neither unlocked nor used before it turns off again.

In the current design, ACTIVE does not mean that the user is definitely performing meaningful activity. Rather, ACTIVE is the complement of the explicit non-active states:

$$\texttt{ACTIVE} = \neg(\texttt{SHUTDOWN} \cup \texttt{IDLE} \cup \texttt{GAP} \cup \texttt{GLANCE})$$

We later combine ACTIVE periods with app activity results to determine whether real user activity occurred.

If a GAP state is present, the user should inspect the logs to determine the likely device state, because in this case the EYES toolbox cannot determine confidently whether the state is SHUTDOWN or IDLE. In general, the user should be alerted if the processing results include too many GAP states.

1. `gap`: Identifies missing-data periods. This step scans the data rows and compares the timestamps of two consecutive rows. If the timestamp difference is greater than a specified gap threshold, a gap is identified. The gap threshold is configurable, and its default value is 3 hours. If a gap has a duration shorter than 3 hours, it will not be detected by default. A good practice is to process the log data multiple times with different parameters and compare the results to choose the best one.

2. `shutdown`: Detects the periods when the device was powered off. This is done by analyzing the events related to device shutdown/startup in the log file.

3. `idle`: Identifies periods of inactivity. This is done by analyzing the events related to screen interactive/non-interactive and keyguard events in the log file.

4. `glance`: Identifies wake sessions in which the device screen becomes interactive but the device is neither unlocked nor used by an app before the screen turns off again (see Glance Logic below). This state is useful because otherwise such periods would be counted as ACTIVE by complement, even though they often do not correspond to real user usage.

5. `blocks`: Categorizes segments and merges them into a single list of blocks in time-increasing order based on the start time of each block.

6. `pickups`: Generates the *pickups* data for the device. The *pickups* data indicates how many times the device was picked up during the observation period. Figure 4 shows the generated pickups data. Rows labeled 'ACTIVE' indicate that the device is being used during the interval from 'start_ts' to 'end_ts'.

4755_P01_Android_Chronicle_Android_Raw_Data_06-05-2025_sorted_uniq_reordered_PIC

start_ts	end_ts	duration_seconds	block_type
2025-03-30 15:18:04.781000+00:00	2025-03-30 15:23:09.264000+00:00	304	IDLE
2025-03-30 15:23:09.264000+00:00	2025-03-30 15:23:34.490000+00:00	25	ACTIVE
2025-03-30 15:23:34.490000+00:00	2025-03-30 15:25:54.231000+00:00	139	IDLE
2025-03-30 15:25:54.231000+00:00	2025-03-30 15:27:50.033000+00:00	115	ACTIVE
2025-03-30 15:27:50.033000+00:00	2025-03-30 15:37:32.878000+00:00	582	IDLE
2025-03-30 15:37:32.878000+00:00	2025-03-30 15:38:02.688000+00:00	29	ACTIVE
2025-03-30 15:38:02.688000+00:00	2025-03-30 15:39:00.358000+00:00	57	IDLE
2025-03-30 15:39:00.358000+00:00	2025-03-30 15:40:00.362000+00:00	60	ACTIVE

Figure 4: EYES processing result: Pickups.

For each identified period, there is an associated 'start time', 'end time', and 'type'. We call such a period a **segment**. For a specific segment type, segments are linked together in time-increasing order by their 'start time'. Note that a 'gap' segment might overlap with a 'shutdown' segment.

In the `blocks` step, we use a more sophisticated algorithm to analyze these segments collectively and identify the state of the device for each period. The outcome of this step is a list of blocks linked in time-increasing order based on the start time of each block. This list captures the device's state changes over time.

7.2.1 Glance Logic

The `GLANCE` state is built from the screen and keyguard events used for IDLE analysis, together with the app foreground events used by the `parser`:

- `SCREEN_INTERACTIVE` (Unknown importance: 15)
- `SCREEN_NON_INTERACTIVE` (Unknown importance: 16)
- `KEYGUARD_HIDDEN` (Unknown importance: 18)
- `ACTIVITY_RESUMED` and `ACTIVITY_PAUSED` (1 and 2; 'Move to Foreground' and 'Move to Background' in older logs)

Wake session. A *wake session* is the interval $[t_s, t_e]$, where t_s is a `Screen Interactive` event and t_e is the first `Screen Non-interactive` event after it. If another `Screen Interactive` event occurs before t_e, the session restarts at that later event. If a SHUTDOWN boundary (`Device`

`Shutdown`, `User Stopped`) or the start of a GAP occurs before t_e, the session is *incomplete* and is not classified. Events with the same timestamp as t_s or t_e belong to the session. If a `Screen Non-interactive` and a `Screen Interactive` event have the same timestamp, the screen went off and straight back on: the first ends the current session and the second starts the next one.

Evidence of use. A wake session shows evidence of use if either of the following holds within $[t_s, t_e]$:

1. a `Keyguard Hidden` event occurs (the device was unlocked); or
2. one or more non-system apps are in the foreground for a total of at least 5 seconds. An app is in the foreground from its `Activity Resumed` event until its `Activity Paused` event, or until t_e if it is not paused first. Only foreground periods that start inside the session are counted.

System packages do not count as foreground apps, because they can come to the foreground while the lock screen is showing: launchers (package name contains `launcher`, or `org.chromium.arc.home`), `com.android.systemui`, `android`, and lock-screen packages (`com.amazon.kindle.kso`). The Amazon Kids home screen (`com.amazon.tahoe`) does count, because it is only shown once the child profile has been entered.

GLANCE. A complete wake session with no evidence of use is labeled `GLANCE`. GLANCE therefore captures the behavioral scenario: the screen lights up, but the user neither unlocks the device nor uses an app before the screen turns off again.

Rationale for rule 2. `Keyguard Hidden` alone is not a reliable signal of use. Some devices, mostly Fire tablets running Amazon Kids, rarely or never log it: in our data (390 files), 24 files log fewer than 2 `Keyguard Hidden` events per 100 `Screen Interactive` events, and 93 more log between 2 and 20. A device's lock-screen settings can also change during a recording that spans several months. On these tablets, wake sessions without `Keyguard Hidden` lasted a median of about 5 minutes, and about 90% of them had an app in the foreground. The same happens on phones that do log `Keyguard Hidden`: in a sample of these phones, 31% of the wake sessions without it had an app in the foreground (for example calls answered from the lock screen, the camera shortcut, or unlock methods that skip the lock screen), and those sessions accounted for 89% of the time in all such sessions. Without rule 2, all of this time would be labeled GLANCE and excluded from ACTIVE.

The 5-second minimum excludes the brief resumes that some apps receive when the screen turns on behind the lock screen. The result is not sensitive to its exact value: with any minimum between 2 and 10 seconds, the sessions that qualify contain 98–100% of the foreground-app time.

Keyguard Hidden after the session. Evidence of use is only checked *within* the wake session. If the screen becomes non-interactive first, the session can be labeled `GLANCE` even if an unlock-

related event is recorded shortly afterward. We keep this rule because it is simple and reproducible, and because such a glance immediately preceding a later ACTIVE period is not expected to materially affect downstream ACTIVE usage analysis.

7.3 Phase C: Application & Event Logic

1. `parser`: Parses app activity events to calculate activity-level app usage. The parser respects the logic structure of an app activity. The structure consists of three events: *Activity Resumed*, *Activity Paused*, and *Activity Stopped*.

   In general, an app might have multiple activities. For example, when a user interacts with an app that has multiple panels, each of which may correspond to a unique activity, an *Activity Resumed* event will be triggered when the user navigates to a new panel.

   If a user launches an App and stays on the first panel A (Activity A), then navigates to a new panel B (Activity B), multiple events will be triggered in sequence, as illustrated in Figure 5. Notice that the three events – Activity Paused (A), Activity Resumed (B), and Activity Stopped (A) – happen at almost the same time (usually within 1 or 2 seconds). We consider the time duration from Activity Resumed (A) to Activity Stopped (A) as the actual usage time for activity A.

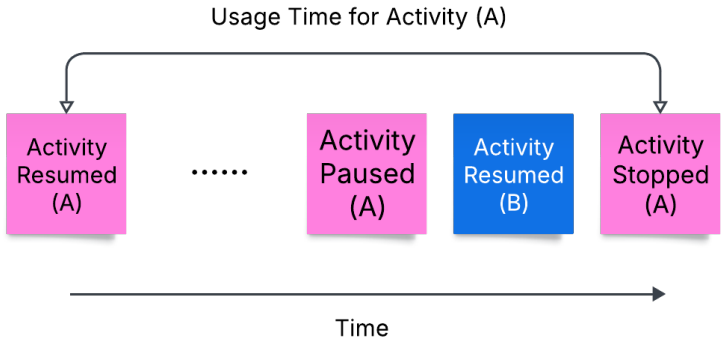


Figure 5: Android Device Event Ordering for Activity A and Activity B.

2. `app`: Calculates app-level usage time.

   Based on the activity-level usage data processed by the `parser`, in this step we merge the activity usage of the same app to produce app-level usage data. The processed data show all apps and their usage durations, sorted by the resumed time (*Activity Resumed*) of each app.

3. `nest`: Performs secondary nested analysis on App usage.

   There are scenarios in which a multi-window app (for example, Youtube) is running while another app is also being used by the user. In this case, both apps may have significant overlapping usage time. We still collect the app-level usage and later identify the overlap so that we do not double count usage time. For a given overlapping interval, the processing result will show that one app is the primary usage and the other app is the secondary usage.

4075_C01_app_1

app_package_name	resume_time	end_time	duration	app_label
com.sec.android.app.camera	2024-09-23 16:26:00.042000+00:00	2024-09-23 16:26:55.008000+00:00	54.966	Camera
com.sec.android.app.camera	2024-09-23 16:26:56.791000+00:00	2024-09-23 16:27:17.925000+00:00	21.134	Camera
com.sec.android.app.camera	2024-09-23 16:27:18.078000+00:00	2024-09-23 16:27:20.238000+00:00	2.16	Camera
com.sec.android.app.camera	2024-09-23 16:27:37.909000+00:00	2024-09-23 16:27:51.912000+00:00	14.003	Camera
com.sec.android.app.camera	2024-09-23 16:28:09.745000+00:00	2024-09-23 16:30:11.115000+00:00	121.37	Camera
com.sec.android.app.camera	2024-09-23 16:30:18.051000+00:00	2024-09-23 16:30:38.483000+00:00	20.432	Camera
com.sec.android.app.camera	2024-09-23 16:30:41.175000+00:00	2024-09-23 16:30:41.408000+00:00	0.233	Camera

Figure 6: EYES processing result: App Level Usage Data.

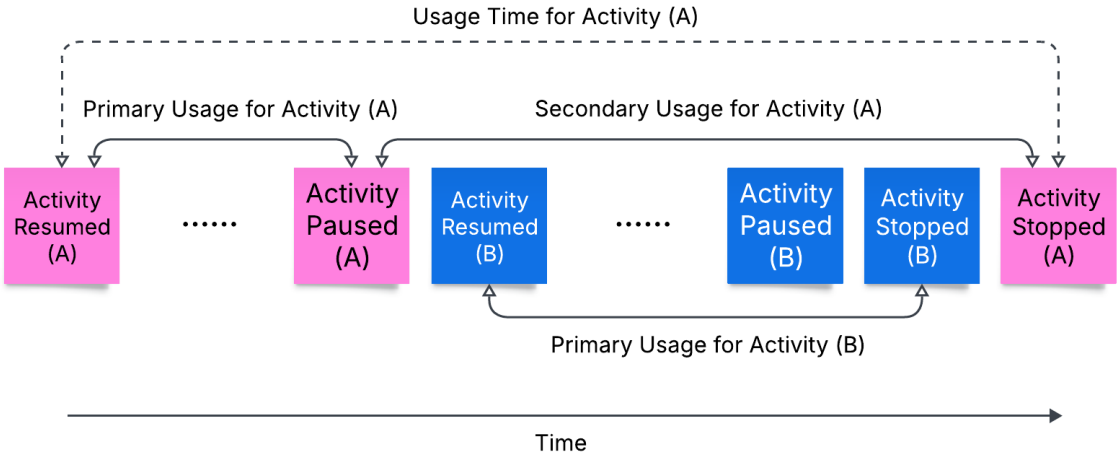


Figure 7: Primary Usage vs. Secondary Usage

Figure 8 shows the data generated by the `nest` unit. There is a 'secondary_duration' column when a given duration is classified as secondary usage for an app.

4. `merge`: Merges device status data with app usage data. This merged dataset has one extra column that labels the device status for each row of app usage data. This extra information is very important for interpreting the usage of a given app. For example, if app usage occurs while the device is in IDLE state, that may indicate that the app was running but the user was not really using it. Figure 9 shows the file format of the merged data.

5. `fau`: Calculates "Final App Usage" (FAU) time, or effective screen time. Once we have the data processed by the `merge` unit, we use a simple approach to sum all app usage time for a given app under the condition that the device status is 'ACTIVE' during the app-usage interval. We refer to this as the FAU time for an app.

   If for some reason a researcher wants to sum an app's usage under device status 'ACTIVE' or 'IDLE', it should be easy to do using the results from the `merge` unit.

4075_C01_primary_secondary_nested

app_package_name	resume_time	end_time	duration	secondary_duration	app_label
com.sec.android.app.camera	2024-09-23 16:26:00.042000+00:00	2024-09-23 16:26:55.008000+00:00	54.966	0	Camera
com.sec.android.app.camera	2024-09-23 16:30:41.175000+00:00	2024-09-23 16:30:41.408000+00:00	0.233	0	Camera
com.sec.android.gallery3d	2024-09-23 16:30:38.193000+00:00	2024-09-23 16:30:41.175000+00:00	2.982	0	Gallery
com.sec.android.gallery3d	2024-09-23 16:30:41.175000+00:00	2024-09-23 16:30:41.408000+00:00	0	0.233	Gallery
android	2024-09-23 16:30:37.902000+00:00	2024-09-23 16:30:38.193000+00:00	0.291	0	Android System
android	2024-09-23 16:30:38.193000+00:00	2024-09-23 16:30:38.256000+00:00	0	0.063	Android System
com.sec.android.app.camera	2024-09-23 16:30:18.051000+00:00	2024-09-23 16:30:37.902000+00:00	19.851	0	Camera
com.sec.android.app.camera	2024-09-23 16:30:37.902000+00:00	2024-09-23 16:30:38.483000+00:00	0	0.581	Camera
com.sec.android.app.launcher	2024-09-23 16:30:15.969000+00:00	2024-09-23 16:30:18.051000+00:00	2.082	0	One UI Home

Figure 8: EYES processing result: Primary and Secondary Usage Data.

4075_C01_primary_secondary_fau

app_package_name	resume_time	end_time	duration	secondary_duration	app_label	device_status
com.sec.android.app.camera	2024-09-23 16:26:00.042000+00:00	2024-09-23 16:26:55.008000+00:00	54.966	0.0	Camera	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:17.823000+00:00	2024-09-23 16:27:18.078000+00:00	0.255	0.0	One UI Home	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:18.078000+00:00	2024-09-23 16:27:18.328000+00:00	0.0	0.25	One UI Home	ACTIVE
com.sec.android.app.camera	2024-09-23 16:27:18.078000+00:00	2024-09-23 16:27:19.744000+00:00	1.666	0.0	Camera	ACTIVE
com.sec.android.app.camera	2024-09-23 16:27:19.744000+00:00	2024-09-23 16:27:20.238000+00:00	0.0	0.494	Camera	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:19.744000+00:00	2024-09-23 16:27:37.909000+00:00	18.165	0.0	One UI Home	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:37.909000+00:00	2024-09-23 16:27:38.556000+00:00	0.0	0.647	One UI Home	ACTIVE
com.sec.android.app.camera	2024-09-23 16:27:37.909000+00:00	2024-09-23 16:27:51.772000+00:00	13.863	0.0	Camera	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:51.772000+00:00	2024-09-23 16:27:58.678000+00:00	6.906	0.0	One UI Home	ACTIVE
com.sec.android.app.camera	2024-09-23 16:27:51.772000+00:00	2024-09-23 16:27:51.912000+00:00	0.0	0.14	Camera	ACTIVE
com.samsung.android.app.galaxyfinder	2024-09-23 16:27:58.678000+00:00	2024-09-23 16:28:09.745000+00:00	11.067	0.0	Finder	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:58.678000+00:00	2024-09-23 16:28:00.286000+00:00	0.0	1.608	One UI Home	ACTIVE
com.sec.android.app.camera	2024-09-23 16:28:09.745000+00:00	2024-09-23 16:30:09.509000+00:00	119.764	0.0	Camera	ACTIVE

Figure 9: EYES processing result: App Usage and Device Status.

7.4 Sessions

A *session* is a period of continuous app use: a chain of apps in the foreground, one app on top at a time, linked by the transitions between them. Sessions are built only from app events. They do not depend on the screen or keyguard events used for the device states (Phase B), so a session can occur while the device is IDLE, for example navigation shown on a car's display while the phone is locked. The two timelines are kept separate and overlaid afterwards: where they agree the result is well supported, and where they disagree the difference points either to a real scenario (such as navigation) or to missing events.

The `sessions` unit reads the reordered log of each raw file. Event names are used in both forms: *Activity Resumed / Move to Foreground*, *Activity Paused / Move to Background*, and *Activity Stopped / Unknown importance: 23*.

Foreground period. App A is in the foreground from its *Activity Resumed* event until the first of the following. The end reason is recorded as the period's *end evidence*.

Event ending the period	End evidence	Note
A's *Activity Paused*	`Paused`	The normal case (observed).
Another app's *Activity Resumed* while A is on top	`next_resumed`	A's *Activity Paused* is missing; only one app can be on top.
A's *Activity Stopped* with no *Activity Paused*, when A has not been resumed in the previous 5 s	`killed`	The app was closed. A *Stopped* soon after a *Resumed* of the same app belongs to the activity the app just left and does not end the period.
Screen off, if A is not paused within 5 s of it	`screen_off`	Normally the app is paused within 1 s of the screen turning off (97–98% on phones).
Device Shutdown or *User Stopped*	`shutdown`	
A boot (*Device Startup* or *User Unlocked*) while A is still on top, more than 1 s after A's last activity	`power_off`	The shutdown was not logged (Section 9.2). An app cannot stay on screen across a reboot, so the period ends when the device went off: at the start of the longest stretch without any event between A's last activity and the boot. Apps that come up within the boot itself (within 1 s of the boot event, as in the GAP unit) are not affected.
End of the log	`end_of_log`	

An *Activity Paused* of A followed within 1 s by an *Activity Resumed* of A is a move between two activities (screens) of the same app and does not end the period.

Link. App B continues the session of app A if B's foreground period starts within 1 second of A's ending, and A did not end at screen off, shutdown, power-off or the end of the log. In our data the median time from *Paused*(A) to *Resumed*(B) is 0.01 s and 98% of transitions take under 1 s, so a longer gap means that no app was on top. How each link was established is recorded:

- `full`: *Paused*(A) $\rightarrow$ *Resumed*(B) $\rightarrow$ *Stopped*(A), the complete pattern described in Phase C (*Stopped*(A) within 5 s).
- `paused_resumed`: *Paused*(A) $\rightarrow$ *Resumed*(B), but *Stopped*(A) was not seen. This is every link in groups A0 and F0 (Section 3), which never log *Activity Stopped*.
- `resumed_only`: *Resumed*(B) while A was on top; A's *Paused* is missing.
- `stopped_resumed`: A was stopped without a *Paused* (killed), then *Resumed*(B).

Only `full` links are fully observed; the others are inferred.

Session. A session is a maximal chain of linked foreground periods. It starts at the first app's *Activity Resumed* and ends at the end of the last app's foreground period. Consecutive periods of the same app are merged into one entry of the chain. Launchers and other system packages (the

same list as in the Glance Logic) stay in the chain, because `launcher > app` and `app > app` are different ways of reaching an app; a session made only of system packages (for example, time spent on the home screen) is flagged.

Output. For each raw file the unit writes two files to `processed/sessions/`:

- `<file>_SESSION.csv`, one row per session: `session_id` (1, 2, 3, ... in time order), `start_time`, `end_time`, `duration_s`, `n_apps`, `first_app`, `last_app`, `chain` (the apps in order, e.g. `launcher > com.facebook.katana > com.android.chrome`), `end_evidence`, `links_full`, `links_inferred`, `system_only`.
- `<file>_SESSION_APPS.csv`, the trigger chain, one row per app in each session: `session_id`, `order`, `app_package_name`, `app_label`, `start_time`, `end_time`, `duration_s`, `n_activities`, `end_evidence`, `link_from_previous`, `system_app`.

Not yet included. Apps that stay visible in picture-in-picture mode after the next app comes to the foreground (Section 3) are currently recorded only while they are on top; their picture-in-picture time is not yet added as secondary use. The overlay of sessions with the device states is a separate step.

7.5 Phase D: Final Outputs

1. `visual`: Generates a visual display of app usage for a given raw file. This helps researchers understand the data more easily. Figure 10 shows an example of app-usage visualization. From the figure, it is easy to get an overview of app usage for each day.
2. `allevents`: Compiles a comprehensive list of all detected events and app usage. This output file includes 1) all activity events, 2) all app usage, and 3) device status.
3. `eventstz`: This unit adds timezone information to the data generated by `allevents`. This is the final unit specified in the Makefile. We can use the commands from Table 1 to run this unit, and it will trigger all earlier units sequentially. Figure 11 shows the format of the data that integrates all activity events, app usage, and device status.

8 Usage Commands

Table 1 lists the commands that a researcher can use to run the EYES toolbox scripts. All commands require the data directory to be specified using the `dir` variable.

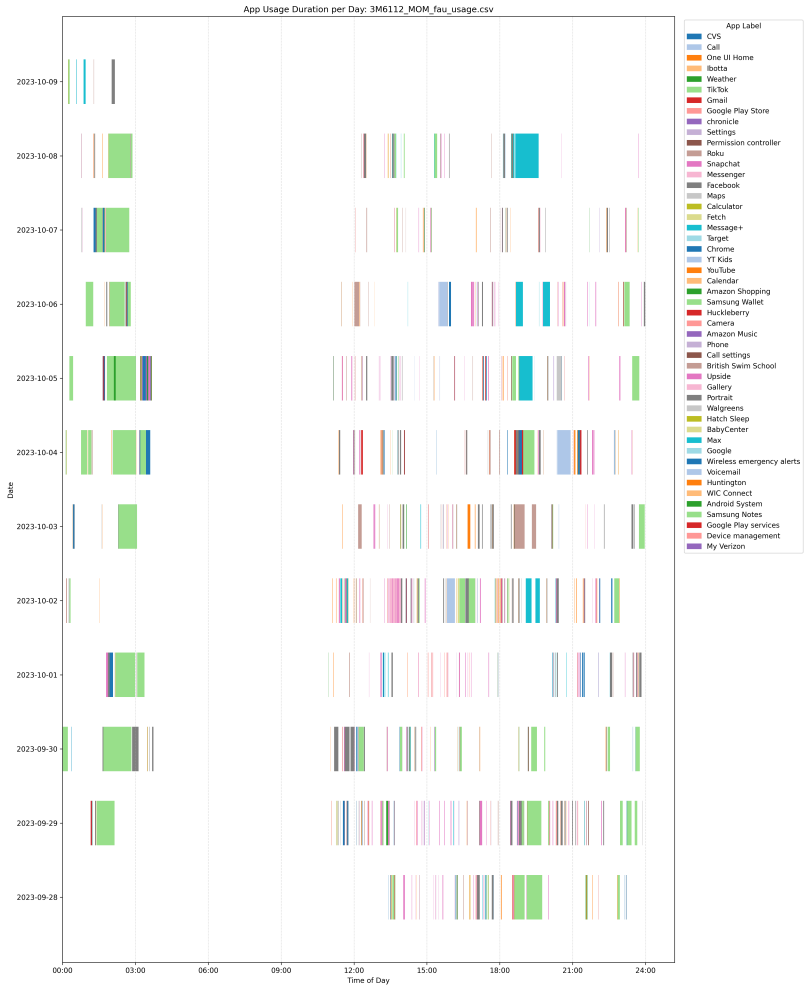


Figure 10: EYES processing result: Visualization of App Usage.

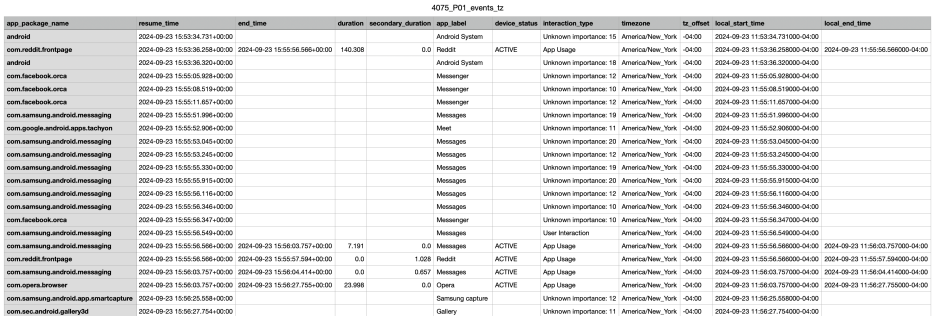
9 Log Events defined in the Android System

Table 2 shows all the events that are defined in the Android System (in the file `UsageEvents.java`):

9.1 Event Classes

Chronicle writes each event either by name (*Activity Stopped*) or, when it has no name for it, by number (*Unknown importance: 23*); the number is the constant in Table 2. In files that switch from one form to the other, the switch is clean: in the A1 data, no event is ever written in its old form after its name has appeared. The toolbox maps both forms, and the old-format *Move to Foreground* and *Move to Background*, to one canonical name.

Every event is then put in one of three classes by what it tells us (Table 3):



4075_P01_events_tz

app_package_name	resume_time	end_time	duration	secondary_duration	app_label	device_status	interaction_type	timezone	tz_offset	local_start_time	local_end_time
android	2024-09-23 15:53:34.731+00:00				Android System		Unknown importance: 15	America/New_York	-04:00	2024-09-23 11:53:34.731000-04:00	
com.reddit.frontpage	2024-09-23 15:53:36.258+00:00	2024-09-23 15:55:56.566+00:00	140.308	0.0	Reddit	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:53:36.258000-04:00	2024-09-23 11:55:56.566000-04:00
android	2024-09-23 15:53:36.320+00:00				Android System		Unknown importance: 18	America/New_York	-04:00	2024-09-23 11:53:36.320000-04:00	
com.facebook.orca	2024-09-23 15:55:05.928+00:00				Messenger		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:55:05.928000-04:00	
com.facebook.orca	2024-09-23 15:55:08.519+00:00				Messenger		Unknown importance: 10	America/New_York	-04:00	2024-09-23 11:55:08.519000-04:00	
com.facebook.orca	2024-09-23 15:55:11.657+00:00				Messenger		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:55:11.657000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:51.996+00:00				Messages		Unknown importance: 19	America/New_York	-04:00	2024-09-23 11:55:51.996000-04:00	
com.google.android.apps.tachyon	2024-09-23 15:55:52.906+00:00				Meet		Unknown importance: 11	America/New_York	-04:00	2024-09-23 11:55:52.906000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:53.045+00:00				Messages		Unknown importance: 20	America/New_York	-04:00	2024-09-23 11:55:53.045000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:53.245+00:00				Messages		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:55:53.245000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:55.330+00:00				Messages		Unknown importance: 19	America/New_York	-04:00	2024-09-23 11:55:55.330000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:55.915+00:00				Messages		Unknown importance: 20	America/New_York	-04:00	2024-09-23 11:55:55.915000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:56.116+00:00				Messages		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:55:56.116000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:56.346+00:00				Messages		Unknown importance: 10	America/New_York	-04:00	2024-09-23 11:55:56.346000-04:00	
com.facebook.orca	2024-09-23 15:55:56.347+00:00				Messenger		Unknown importance: 10	America/New_York	-04:00	2024-09-23 11:55:56.347000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:56.549+00:00				Messages		User Interaction	America/New_York	-04:00	2024-09-23 11:55:56.549000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:56.566+00:00	2024-09-23 15:56:03.757+00:00	7.191	0.0	Messages	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:55:56.566000-04:00	2024-09-23 11:56:03.757000-04:00
com.reddit.frontpage	2024-09-23 15:55:56.566+00:00	2024-09-23 15:55:57.594+00:00	0.0	1.028	Reddit	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:55:56.566000-04:00	2024-09-23 11:55:57.594000-04:00
com.samsung.android.messaging	2024-09-23 15:56:03.757+00:00	2024-09-23 15:56:04.414+00:00	0.0	0.657	Messages	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:56:03.757000-04:00	2024-09-23 11:56:04.414000-04:00
com.opera.browser	2024-09-23 15:56:03.757+00:00	2024-09-23 15:56:27.755+00:00	23.998	0.0	Opera	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:56:03.757000-04:00	2024-09-23 11:56:27.755000-04:00
com.samsung.android.app.smartcapture	2024-09-23 15:56:25.558+00:00				Samsung capture		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:56:25.558000-04:00	
com.sec.android.gallery3d	2024-09-23 15:56:27.754+00:00				Gallery		Unknown importance: 11	America/New_York	-04:00	2024-09-23 11:56:27.754000-04:00	

Figure 11: EYES processing result: All events, App Usage, and Device Status.

Table 1: Major commands to run the EYES toolbox scripts

Full Process	`make all dir=/path/to/data`	Run the entire pipeline (Visuals + Pickups)
GUI Launcher	`make gui`	Launch the EYES desktop GUI. Select a raw-data folder and click `Go` to run the pipeline.
Clean Up	`make clean dir=/path/to/data`	Delete all the processed results (to start over)
Specific Unit	`make [target] dir=/path/to/data`	Run up to a specific target (e.g., `make app`)
The final unit	`make eventstz dir=/path/to/data`	Run timezone column adjustment. This will also run all the units ahead sequentially.

- **Usage**: a person did something. An app came to the foreground or left it, the device was unlocked, the person interacted with an app or a notification.
- **State**: the device changed state. Screen on or off, lock screen shown, power off or on, configuration change.
- **Heartbeat**: the device was on and logging, and nothing more. These are background events that happen whether or not anyone uses the device: standby bucket changes, incoming notifications, foreground services. Codes that Android does not define (32, 10001, 10002) are counted as heartbeats.

Heartbeats as evidence. Heartbeats were ignored in earlier versions of this manual. They matter when the key events are missing: a lost *Screen Non-interactive* or *Device Shutdown* leaves the device-state timeline wrong, and the heartbeats are then the only record of what the device was doing. Two examples of what they show:

- *Notification Interruption* (12) is written when an app posts a notification that plays a sound, vibrates or flashes the LED (Android's `UsageEvents.java`: "an app posted an interruptive notification"). It is written under the posting app and needs no action from the person. In

Table 2: Andorid System Events and Numbering

Event Name	Unknown importance #
ACTIVITY_RESUMED	1
ACTIVITY_PAUSED	2
END_OF_DAY	3
CONTINUE_PREVIOUS_DAY	4
CONFIGURATION_CHANGE	5
SYSTEM_INTERACTION	6
USER_INTERACTION	7
SHORTCUT_INVOCATION	8
CHOOSER_ACTION	9
NOTIFICATION_SEEN	10
STANDBY_BUCKET_CHANGED	11
NOTIFICATION_INTERRUPTION	12
SLICE_PINNED_PRIV	13
SLICE_PINNED	14
SCREEN_INTERACTIVE	15
SCREEN_NON_INTERACTIVE	16
KEYGUARD_SHOWN	17
KEYGUARD_HIDDEN	18
FOREGROUND_SERVICE_START	19
FOREGROUND_SERVICE_STOP	20
CONTINUING_FOREGROUND_SERVICE	21
ROLLOVER_FOREGROUND_SERVICE	22
ACTIVITY_STOPPED	23
ACTIVITY_DESTROYED	24
FLUSH_TO_DISK	25
DEVICE_SHUTDOWN	26
DEVICE_STARTUP	27
USER_UNLOCKED	28
USER_STOPPED	29
LOCUS_ID_SET	30
APP_COMPONENT_USED	31

Table 3: Event classes

Class	Events (codes)
Usage	Activity Resumed (1), Activity Paused (2), Activity Stopped (23), Keyguard Hidden (18), User Interaction (7), Notification Seen (10), Shortcut Invocation (8), Chooser Action (9)
State	Screen Interactive (15), Screen Non-interactive (16), Keyguard Shown (17), Configuration Change (5), Device Shutdown (26), Device Startup (27), User Unlocked (28), User Stopped (29)
Heartbeat	everything else, e.g. Standby Bucket Changed (11), Notification Interruption (12), Foreground Service Start/Stop (19/20), Slice Pinned (13/14)

the A1 data, 43% (parents) and 59% (children) of these events fall while the screen is off, and the screen turns on within 10 s after only 5% of them.

- In IDLE periods of one hour or more, where the device is known to be on, 98% contain heartbeats (parents about 11 per hour, children about 2 per hour). In SHUTDOWN periods of one hour or more, 99% contain none. Of the stretches of three hours or more with no usage or state event, 93% contain heartbeats: the device was on but nobody used it.

A heartbeat is strong evidence that the device was on. Its absence is weak evidence that it was off: a sleeping device can go hours without one (in IDLE periods of one hour or more, the longest wait between heartbeats has a median of 0.7 h and a 99th percentile of 10.7 h). The toolbox therefore applies **no threshold** to heartbeats. It reports them as evidence for manual review: for each session, the longest stretch with no usage or state event, the number of heartbeats in that stretch, the counts of events by class, and the last usage event.

The classes are implemented in `eyeslib/core/event_classes.py`.

9.2 Shutdown, Startup and User Unlocked

SHUTDOWN periods are built from *Device Shutdown* / *User Stopped* (the device goes off) and *Device Startup* / *User Unlocked* (it comes back). Android writes *User Unlocked* when the user is unlocked for the first time after a boot (the moment the encrypted storage becomes available), so it is a second, independent record of every boot. Table 4 shows how reliable these events are in the A1 data (201 files).

What the toolbox does with them:

- **Both markers are boot events.** *Device Startup* and *User Unlocked* within the proximity window are merged into one boot. A boot with no shutdown before it ends a SHUTDOWN period whose start is taken from the GAP that ends at the boot: the device was off from the last event before the boot. This covers the 512 boots whose startup was lost.
- **A dark stretch whose next event is a boot** is therefore SHUTDOWN: of the 1,080 such stretches with no shutdown at their end, 1,078 are SHUTDOWN; the other two were caused by the GAP bug below.

Table 4: Shutdown, startup and User Unlocked events (A1)

Device Shutdown events	3,322
Device Startup events	2,467
User Unlocked events	2,773
Device Startup events with a *User Unlocked* within one minute (almost always in the same second, written a few milliseconds earlier)	98.2%
Boots with a *User Unlocked* but no *Device Startup* logged (the startup event was lost)	512 (18.5%)
Dark stretches (one hour or more with no event of any kind)	26,431 (242,518 h)
whose next event is a boot marker (within a minute)	1,104 (37,653 h)
with no shutdown logged at their end: the device was off	1,080
ending in a logged shutdown and boot (a restart)	24
with a boot marker 1 min to 1 h later, after other events	77
with a boot marker only hours later (a separate reboot)	726
with no boot marker after them	24,524

- **A restart after a long dark stretch is not proof that the device was off.** In 24 dark stretches (3,969 h) the stretch ends in a logged *Device Shutdown* followed about 40 s later by the boot: a normal restart at the end. It proves the restart, not what the device did before it, so the stretch stays GAP (or IDLE).
- **Most dark stretches have no boot marker.** The device was either asleep (it can go hours without any event, Section 9.1), or off with the whole shutdown–boot sequence lost from the log. The log cannot tell these apart; they are left to manual review, with the heartbeats and the device's own longest wait between heartbeats while on as evidence.

No threshold on the length of a dark stretch is used: a two-hour rule ("no event for two hours means off") would label as off most nights of devices that only sleep, and 183 of 198 devices go two hours or more between heartbeats while on.

GAP detection fix (2026-09-30). The GAP unit appended the log row itself to its output and then set its label. When a boot event followed a gap within a second, the same row was relabelled from *Gap End* to *Gap Start*, which broke the Start/End pairing in 89 of 201 A1 files; a second flaw (an out-of-date previous event after a shutdown) created overlapping duplicate gaps. Both are fixed: rows are copied before labelling, the previous event is updated, and exact duplicate gaps are dropped. Effect on A1: all GAP files are well-formed; SHUTDOWN +10 h (one file), GAP −447 h (36 files, overlapping duplicates removed); IDLE and GLANCE unchanged.

10 Device State Analysis

Before any app is analysed, the toolbox decides what the device was doing at every moment of a log: **Off** (shut down), **Idle** (on, screen off) or **Active** (on, screen on). Only Active time can be screen time. Because the logs lose events (Sections 9.1 and 9.2), the analysis does not guess a single state. It starts from what the log confirms, adds evidence step by step, and keeps track of what it still cannot tell.

10.1 Framework

The log is partial, so the analysis starts from what it gives for certain and solves the rest from there.

Anchors. Three kinds of evidence are taken as confirmed:

- *device state*: a logged shutdown or startup (Off), a logged screen-on or screen-off (Active, Idle);
- *app usage*: *Activity Resumed*, *Paused* and *Stopped* of a named app;
- *signs of a device that is not silent*: the boot storm (a boot happened), background events with the screen off (the device is on and Idle).

Two problems, in order. (1) Solve the device state for the whole file: every moment is Off, Idle or Active. The anchors settle part of it, rules extend them into the gaps, and what cannot be settled stays marked unsure. (2) Solve the app usage for the whole file: inside Active time only, each moment is given to the app in front, or to no app. This section covers the first problem.

Principles.

- Evidence only narrows what a stretch can be; a weak sign never overrides a strong one.
- Every rule comes from a case found in review and is measured on all files before it is kept. Signs that do not hold up are discarded (Section 10.3).
- When the log gives no basis, nothing is granted. Screen time is therefore a lower bound wherever the log has gaps.
- Screen time is use of the device's own screen, counted whenever the screen is on with an app in front; whether a person is watching cannot be told. Apps shown on another screen (a car display) are a separate category.
- The two problems are linked: app evidence helps settle the device state (the same app before and after a silence), and the device state bounds app usage (no app time outside Active).

10.2 Model

The timeline of a log is cut at every logged event. Each interval between two consecutive events is a *quiet stretch*: a gap with no event at all. Each gets the *set* of states it could be in (Table 5).

Table 5: Device-state labels

Label	Possible states	Meaning
Off	O	shut down
Idle	I	on, screen off
Active	A	on, screen on
Unsure OI	O or I	screen off; the device may have been shut down
Unsure IA	I or A	on; the screen state is unknown
Unsure OA	O or A	screen on; the device may have been shut down
Unsure 3	O, I or A	nothing is known

Evidence only *removes* states from a set, so a weaker rule cannot overturn a stronger one. When one state is left, the stretch is confirmed. The analysis runs in two steps: first *Off or On* (Section 10.3), then, for the time the device was on, *Idle or Active* from the screen events (Section 10.6). Consecutive intervals with the same label form a *block*, used for the statistics; a block can contain events. The unit of review is the quiet stretch (Section 10.7).

Tolerance (5 minutes). A quiet stretch shorter than 5 minutes is never counted as possibly Off. If that is wrong, it is wrong by at most 5 minutes, which does not matter for the study. (A reboot can be faster: the fastest logged in A1, *Device Shutdown* to *Device Startup* with nothing in between, took 10.3 s; 1% of logged reboots took under 31 s, the median about 20 minutes.) Only stretches of 5 minutes or more are examined in step 1 and listed for review.

Two layers. The rules are applied under two sets of assumptions:

- **Strict**: a stretch is Off only when a logged boot event says so (shutdown, startup, User Unlocked). It is *not* Off when an On sign says so. With neither, it stays possibly Off.
- **Likely**: a stretch is also Off when the boot signature says so, or when it is a silence of 6 hours or more. A stretch with no Off sign is taken as not Off.

The strict layer is what the logs support with direct evidence; the likely layer is the main result.

10.3 Step 1: Off or On

Whether a quiet stretch was Off is decided by the events right around it, mostly the events right after it: a phone coming back from Off leaves a different trail from a phone that only slept. Each stretch keeps the sign that decided it (Table 6). "Right before" and "right after" mean within 60 s (the boot window, the same as in the session builder); for a boot sign, the stretch concerned is the longest silence ending in the 60 s before the sign, so that heartbeats running up to a boot stay On.

Normal unlock. After every boot the first unlock is logged as *User Unlocked*. So when the next unlock after a stretch is an ordinary *Keyguard Hidden*, no boot happened in between and the device

Table 6: Signs used in step 1

Sign	Evidence	Says
`shutdown_before`	a *Device Shutdown*, then the longest silence before the next usage event or boot	Off, both layers
`startup_after`	a *Device Startup* right after	Off, both layers
`shutdown_startup`	both of the above on the same stretch	Off, both layers
`unlocked_after`	a *User Unlocked* right after (Android writes it at the first unlock after every boot)	Off, both layers
`boot_signature`	the events right after match the device's boot signature (below)	Off, likely
`long_quiet`	the stretch is 6 hours or more (below)	Off, likely
`normal_unlock`	the next unlock after the stretch is an ordinary *Keyguard Hidden*, with no *User Unlocked* or *Device Startup* up to 60 s after it	On, both layers
`app_continuity`	the same user app on both sides of the stretch	On, both layers

stayed on. Unlike continuity, this does not depend on which app was open. It was checked on the 1,702 quiet stretches of A1 that end in a logged *Device Startup*: none is followed by an ordinary unlock. The test must compare *times*, not the order of rows: at a boot, *Keyguard Shown*, *Screen Interactive*, sometimes a *Keyguard Hidden*, and *User Unlocked* carry the same timestamp, and by row order a *Keyguard Hidden* appears to precede the *User Unlocked* in half of the boots.

Continuity. When the last Resumed/Paused before a stretch and the first after it belong to the same *user* app, as *Paused(X) . . . Resumed(X)* or *Resumed(X) . . . Paused(X)*, the device did not go off: a reboot restarts at the lock screen and the home screen. Only user apps are evidence. Excluded (`NON_USER_APP`): the logger (Chronicle); Android system parts and services (`android`, System UI, Settings, installer, permission controller, setup wizard, Google Play services and Family Link supervision, screen savers, software updaters, background services); home screens and their feeds (launchers, Google Discover, carrier feeds such as MetroZONE and Sliide); and apps the phone brings to the front by itself (call screen, clock and alarm apps, Android Auto). An alarm resumes the clock app in the morning whether or not the phone rebooted overnight, so it is no evidence. Of 3,517 logged reboots in A1, only 31 (0.9%) have the same app on both sides, none after an hour or more off.

A discarded sign: a system app pausing first. A system app (Settings, home screen, System UI) logging *Paused* without having started was first used as a boot sign. It did not hold up: in A1 it contradicted a normal unlock in 903 stretches and stood alone in only 44, and a phone waking from sleep produces the same pattern. It is no longer used (decided by the reviewer). With it, the category of *conflicts* (an Off sign and an On sign on one stretch) disappeared: it was the only sign that produced them.

Long silence. A normal unlock afterwards is weak evidence for a long stretch. Reviewing the stretches of 12 hours or more that the signs called On (507 stretches, 140,148 h; many of them months long) showed why: the logging itself had stopped and restarted. In one log (4520_P01) a logged shutdown is followed an hour later by a boot storm, the log stops in the middle of that boot, and resumes 139 days later like a phone already running, with an ordinary unlock 13 minutes on. "No boot marker afterwards" was read as "the device stayed on", but a boot inside the gap would

have left no marker either. The deciding unlock can also lie hours after the stretch. A phone that is on rarely goes long without one event (99% of its waits between background events are under 10.7 h). The reviewer therefore set these rules for the likely layer:

- a silence of 12 hours or more is Off;
- a silence of 6 to 12 hours is Off; when the screen was not Idle going into it (Section 10.6), it is also listed for review, because it is the only kind that could hide screen time.

The reasoning: such a silence is never counted as Active, so calling it Off instead of Idle cannot change screen time. The cost is in the meaning of Off. For these stretches it reads *off or not logging*, and an ordinary night of 6 hours without an event is included; the events cannot tell these apart. Off by logged events and Off by long silence are therefore reported separately (Table 8), and the Off and Idle totals must not be read as "switched off" and "asleep". In the strict layer these stretches stay possibly Off.

10.4 Boot signature

A boot whose *Device Startup* and *User Unlocked* were both lost can still be recognised by what else the device logs while starting. The method learns, from the boots that *are* confirmed, what a boot looks like on each device, and then looks for the same pattern after other quiet stretches.

Why a signature. When Android starts, it re-assigns the standby bucket of the apps on the device, which appears as a burst of *Standby Bucket Changed* events (a "boot storm"). A count of such events is not enough: A1 has 4,374 bursts of 20 or more, and only 59% of them are at a logged boot; the others are small batches (median about 30 apps) that Android re-sorts during normal use. The share of the device's apps in the burst is better (at least half of the apps: 62% of logged boots, 0.03% of ordinary wake-ups), but *which* apps are involved is better still: a boot touches system packages that almost never get a standby-bucket change otherwise.

Definition.

- *Confirmed boot*: a *Device Startup* or a *User Unlocked* (markers less than 5 minutes apart are one boot).
- *Boot phase*: the events in the 60 s around the marker. *Item*: an event type together with its app, for example "Standby Bucket Changed, `com.android.phone`". Only background events (the heartbeat class, Section 9.1) are items: unlocks, app switches and screen events also happen at every ordinary wake-up and do not tell a boot apart.
- *Signature of a device*: the items present in at least 80% of its confirmed boot phases. A device has its own signature when it has at least 5 confirmed boots and the signature has at least 5 items.

- *Score* of a quiet stretch: the share of the signature items present in the 60 s after it. A score of 0.8 or more is a boot.
- *Pooled signature*, for devices without their own: the items in the signature of at least half of the devices that have one; a score of 0.5 or more is a boot.

Validation. Each confirmed boot was scored against a signature built from the device's *other* boots (leave-one-out), and compared with ordinary wake-ups: the first 60 s after a quiet stretch of 5 minutes or more with no boot marker or shutdown nearby (Table 7). The two groups hardly overlap: boots score near 1 (median 0.99), ordinary wake-ups near 0 (median 0.00), so the result does not depend on the exact cut-off. Signatures of fewer than 5 items are not used: with one to four items anything can match, and they caused 271 of the 280 false hits before the minimum was set. The pooled signature was tested by leaving out each device in turn (all exports of the same participant): boots score 0.91 (median), and 99.9% of ordinary wake-ups score under 0.09.

Table 7: Boot signature: validation on A1

	Confirmed boots	Ordinary wake-ups
Device's own signature (at least 5 items), leave-one-out, score ≥ 0.8		
Scored	2,381	258,352
Recognised as a boot	2,201 (92.4%)	9 (0.003%)
Pooled signature (from the other devices), score ≥ 0.5		
Scored	2,761	76,758
Recognised as a boot	79.6%	6 (0.008%)
For comparison: at least half of the device's apps in a burst		
Recognised as a boot	62%	0.03%

What the signatures contain. In A1, 124 of the 201 logs have their own signature (typically about 100 items); the other 77 use the pooled one. The items are almost all (98%) standby-bucket changes. The pooled signature has 22 items, all standby-bucket changes of Android system packages: `android`, phone, telephony and MMS providers, Bluetooth, keychain, input devices, location, network stack, Settings and settings provider, shell, and others. These packages get a standby-bucket change at boot and almost never otherwise.

What it finds. Applied to all 671,284 quiet stretches of 5 minutes or more in A1, the signature recognises only 9 boots that have no marker (836 h). So when a boot's storm is in the log, its markers almost always are too: boots with lost markers are rare. This supports the likely layer's assumption with data. The signature cannot find a boot that happened while nothing was being logged.

An unlock after a boot is not an On sign. After a boot the person unlocks the device, which is logged as *Keyguard Hidden*. *User Unlocked* is a separate system event that normally comes with the boot; when it is lost, the unlock looks ordinary. In 7261_C01 a silence of 16.5 days is followed by a boot storm, screen on, lock screen and a *Keyguard Hidden* 17 s later, with neither *Device Startup* nor *User Unlocked*. So when the signature fires, the unlock that follows does not count as an On

sign and there is no conflict. This case also shows that the normal-unlock sign can be wrong when *User Unlocked* is lost.

Shutdowns leave no storm. The storm belongs to booting only. Of the 3,322 logged *Device Shutdown* events in A1, about 3% have 20 or more standby-bucket changes in the minute before (median 0); of the 2,467 *Device Startup* events, 79% have 20 or more in the 10 s before the marker (median 115). What precedes a shutdown is apps closing (*Activity Stopped* or *Paused* within a tenth of a second), which also happens at an ordinary screen-off; the shutdown event itself is often logged twice (852 times). The method is therefore one-directional: it can recover a boot whose markers were lost, not a lost shutdown.

10.5 Step 1 results (A1)

Table 8 gives the verdict for all quiet stretches of 5 minutes or more, with the sign that decided it.

Table 8: Off or On: quiet stretches of 5 minutes or more in A1 (671,284 stretches, 373,793 h)

Verdict and sign	Stretches	Hours
Off	**3,521**	**197,847**
shutdown + startup	1,570	4,093
startup after	132	4,416
shutdown before	122	4,804
User Unlocked after	225	25,105
boot signature	9	836
long silence, 12 h or more	616	158,594
long silence, 6 to 12 h	847	
On	**652,655**	**167,124**
normal unlock (with or without continuity)	643,492	164,021
continuity only	9,163	3,103
Unsure (no sign)	**15,108**	**8,822**

Off by logged events or by the boot signature is 2,058 stretches (39,254 h). Off by long silence is 1,463 stretches (158,594 h): few stretches, but four fifths of all Off time. Of these, 12 (6 to 12 hours, screen not Idle before) are listed for review.

10.6 Step 2: Idle or Active

For the time the device is on, the screen events decide. A *screen run* lasts from one screen event to the next: *Screen Interactive*, *Screen Non-interactive*, *Keyguard Hidden* when the screen was not logged on (an unlock proves the screen was on: the screen-on was lost), or a shutdown or boot, which reset the state (Table 9).

The 30-minute screen-on limit. A screen turns itself off after a timeout, usually between 30 seconds and 10 minutes, and apps that hold the screen on (video) still log events now and then.

Table 9: Screen rules

Rule	Condition	States
`screen_on`	screen on, run closed by a screen off	A
`screen_on_active`	screen on, the screen-off was not logged (the next screen event is another screen on, a boot or the end of the log): up to the last usage event of the run	A
`screen_on_tail`	... and after that event: the screen went off somewhere in it, or the person kept watching	I or A
`screen_on_long_silence`	a silence of more than 30 minutes while the screen is logged on	I or A
`screen_on_same_app`	... unless the same user app is resumed before it and paused after it	A
`screen_on_end_lost`	... a silence of more than 30 minutes that starts at that last usage event	I
`screen_on_tail_idle`	... and from 5 minutes after it, with only background events since	I
`screen_off`	screen off, run closed by a screen on, boot or shutdown	I
`screen_off_lost_on`	screen off, followed by another screen off: the screen-on was not logged	I or A
`screen_off_until_use`	... but from that screen off up to the first usage event	I
`background_only`	where a screen-on was lost or the state is unknown: from 5 minutes after the last usage event, with only background events since	I
`screen_unknown`	before the first screen event, or after a boot, shutdown or long silence until the next one	I or A

A silence of more than 30 minutes while the screen is logged on most likely hides a lost screen-off, and is not counted as Active.

A long silence breaks a screen-on run. A screen-off can be logged months after the screen-on, across a silence in which nothing was recorded (4088_P01: 135 days). The screen did not stay on across it. A silence of 6 hours or more therefore ends the run: up to the silence the screen-off counts as lost, and after it the screen state is unknown until the next screen event.

Idle 5 minutes after the last use. When the screen-off was lost, the device is taken as Idle once only background events have been logged for 5 minutes after the last usage event (limit set by the reviewer; screens time out within minutes). In 4088_P01 the last usage event is a *Notification Seen* 12.7 minutes before the silence, followed only by incoming notifications.

Idle until the first use. Two screen-offs in a row mean that a screen-on was lost between them. It cannot have come before the person started using the device again, so from the first screen-off up to the first usage event the device is Idle; only the rest, up to the second screen-off, stays unsure (9124_P01: 62 minutes of background events after a screen-off, then a silence of 11.8 hours, then use).

Use that ends inside a silence. In 4090_P01 the person unlocks the phone with YouTube in front at 00:31:09. The next event is an SMS notification at 06:00:09, followed by background events only and a screen-on at 07:03. YouTube started at the beginning of the silence and ended somewhere inside it, and that moment was not logged: it could be two minutes or two hours. Since

the log gives no basis for any amount, no screen time is granted: a silence of more than 30 minutes that starts at the last sign of use and ends with the screen off is Idle (decided by the reviewer). This is a known source of undercounting, for video in particular, which generates no events while it plays. In A1 it concerns 62 stretches (123 h). It does not apply when the screen-off after the silence *is* logged; those are covered next.

An app in front across a silence. The opposite case, in the same log: YouTube is resumed at 03:13 with the screen on; the next events are notifications at 04:15 and 06:40; at 07:04 YouTube is paused and the person makes a call; the screen-off is logged at 07:26. An app can only be paused if it was still in front, and a pause is logged whenever the screen goes off, so YouTube was in front with the screen on from 03:13 to 07:04. A silence of more than 30 minutes with the screen on is therefore Active when the same user app is resumed before it and paused after it. It counts as screen time whether or not anyone was watching (decided by the reviewer; Section 10.1). In A1 this is 1,793 silences (1,323 h), the longest 3.4 hours; YouTube (523 h), Facebook (216 h), YouTube Kids (207 h), Disney+ and Netflix lead. These stretches are kept in a separate list, *Long stretch app usage*, so that they can be checked (Section 10.7).

What counts as a sign of use. Several rules look for the last or the first sign of use. Three points were settled from review:

- *Notification Seen* means a notification became visible on the display, not that it was tapped. It is tied to the screen: of 1,336,855 such events in A1, 98.3% fall while the screen is logged on and 0.5% while it is logged off (most of those within 10 s of the screen-off). *Notification Interruption*, the notification arriving, falls 54.5% on and 44.0% off. So *Notification Seen* is good evidence that the screen was on and weak evidence that a person was looking.
- Lock-screen feeds are not use. In 4111_P02, Metro Play (`com.metro.minus1`, a carrier content feed) logs *Resumed*, *Paused* and *Stopped* within a fraction of a second, every few minutes, while the screen is off, and comes to the front whenever the display wakes: 13,324 resumes in one log. Events of such feeds (Metro Play, MetroZONE) are not counted as signs of use in the screen rules. Time with the feed in front on a lit screen still counts as screen time of that app.
- A pause or stop is not the start of use. The screen-off itself causes one, and a pause with no matching start shows that events were lost, not when the screen came on. After a logged screen-off the device stays Idle until something starts: an app resumed, a user interaction, a notification seen, an unlock. In 4111_P02 two silences in one afternoon each end with a pause that has no start (the home screen, then YouTube Music): the phone was used inside them and none of it was logged.

Background events only. The same 5-minute limit applies wherever the screen state is unsure because a screen-on was lost or the state is unknown: once only background events have been logged for 5 minutes since the last usage event, the device is Idle. This only turns unsure time into Idle; it never changes Active. In 4103_C01 the last usage event is 23.9 hours before a silence of 5.5

hours, with only notifications in between and after. In A1 the rule settles 2,137 h and takes 3,745 stretches off the review list.

A silence between two Idle periods. A silence under 6 hours with Idle before it is Idle: the screen was off going in, and nothing in the log turns it on. The device may have been asleep (Android holds back background work on a phone lying still, so hours without an event are normal), off, or not logging; none of these changes screen time. From 6 hours on, the long-silence rules of Section 10.3 apply instead (decided by the reviewer). If the log was missing during a silence, use inside it cannot be seen under any label: screen time is a lower bound wherever the log has gaps.

10.7 Reviewing quiet stretches

The review list holds quiet stretches, not blocks: each item is one gap between two consecutive events, of at least 5 minutes, so a ruling covers exactly the silence. (A block is not suitable: it can end in events, for example heartbeats that prove the device was on, and an Off ruling on the block would cover them too.) A stretch is listed for one of these reasons:

- **unsure**: its state is not settled in the layer;
- **long Off check**: Off as a silence of 6 to 12 hours, but the screen was not Idle when it began, so it could hide screen time;
- **continuity check** (strict layer): settled by continuity alone and longer than 30 minutes.

Separate from the review list is the list **Long stretch app usage**: the silences of more than 30 minutes that count as Active because the same app is in front before and after. They are not review items, but each can be opened and ruled if something looks wrong.

The review app (tab *Device state review*) has four views: *File by file* (the files with stretches to review, with a count of what is left; a file opens its stretches in time order, with a switch to longest first), *All stretches* (all files together, longest first), *App usage* (the separate list above) and *Day charts*. Filters: layer, label, reason, role, length and status. A stretch shows its label, why it is listed, its Off sign, On sign and boot-signature score, the last event before it and the first after it, and the events around it (earlier and later events can be loaded). The reviewer rules it Off, Idle, Active or still unsure, with a note, one at a time or, as a quick action, many at once (select the stretches, choose the ruling, confirm). Rulings are kept when the data is rebuilt. A *Files* view shows each file as a chart of days (0–24 h, coloured by label).

10.8 Results (A1)

Table 10 gives the hours of the 201 A1 logs (464,479 h of logged timeline) and Table 11 the review list.

Table 10: Device states in A1: hours and share of the logged timeline

Label	Strict		Likely	
Off	38,442	8.3%	197,872	42.6%
Idle	188,660	40.6%	197,327	42.5%
Active	68,020	14.6%	68,131	14.7%
Confirmed	**295,122**	**63.5%**	**463,330**	**99.8%**
Unsure OI	149,881	32.3%	0	0%
Unsure IA	1,106	0.2%	1,149	0.2%
Unsure OA	112	0.0%	0	0%
Unsure 3	18,258	3.9%	0	0%

Table 11: Review list in A1 (quiet stretches of 5 minutes or more)

	Strict	Likely
Review list	19,472	1,648
unsure	18,128	1,636
long Off check	–	12
continuity check	1,344	–
Long stretch app usage (separate list)	1,793	1,793

In the likely layer 99.8% of the timeline is settled. Off in this layer is mostly long silence (off or not logging), so the layers differ in Off and agree in Active. What remains unsure is Idle-or-Active (1,149 h, 0.2%): silences of more than 30 minutes with the screen logged on and no same-app match (521 h), unknown screen state (382 h), time after a lost screen-on (185 h), and the 5 minutes after the last use when the screen-off was lost (60 h). In the strict layer the long silences stay possibly Off, which is why it confirms less.

How much of Active time is silence. A silence labelled Idle cannot reach any app's usage; a silence labelled Active goes to the app in front. Table 12 splits Active time by the length of the gap between two events. Gaps under 5 minutes are ordinary use. Silences over 30 minutes are Active only under the same-app rule. The band in between (26% of Active time) is Active because a logged screen-on and a logged screen-off bracket it. Within that band, 74.4% of the hours have the same user app resumed before and paused after, 3.0% the same non-user app (a call screen, a home screen), 15.1% an app resumed before and something else after (an app event was lost), and 7.5% no app in front (mostly home screens; also Android Auto and Maps, which point to a car display and belong to the second problem).

Table 12: Active time in A1 by the length of the gap between two events (likely layer)

Gap	Intervals	Hours	Share
under 1 min	25,324,041	20,700	30.4%
1 to 5 min	810,932	28,138	41.3%
5 to 10 min	98,074	11,239	16.5%
10 to 30 min	26,748	6,559	9.6%
30 to 60 min	1,739	1,133	1.7%
over 60 min	282	363	0.5%

Implementation. `eyeslib/states/device_state.py` holds the rules and constants (`TOLERANCE_SECONDS` = 300, `BOOT_BURST_SECONDS` = 60, `SCREEN_ON_MAX_SILENCE_SECONDS` = 1800, `CONTINUITY_REVIEW_SECONDS` = 1800, `LONG_QUIET_OFF_SECONDS` = 21600, `LONG_QUIET_SURE_SECONDS` = 43200, `SCREEN_TAIL_IDLE_SECONDS` = 300; `SIG_MIN_BOOTS` = 5, `SIG_MIN_ITEMS` = 5, `SIG_CORE_SHARE` = 0.8, `SIG_SCORE` = 0.8, `SIG_POOL_SHARE` = 0.5, `SIG_POOL_SCORE` = 0.5; `NON_USER_APP`). `eyeslib/runners/run_boot_signatures.py` learns the signatures of a processing group (`processed/states/<stem>_BOOTSIG.json` and `<group>/_states/boot_signature_pooled.json`); `eyeslib/runners/run_device_states.py` then writes, per log, `<stem>_STATES.csv` (the interval table), `<stem>_QUIET.csv` (the stretches to review, with their signs) and `<stem>_SIGNS.csv` (the step-1 verdict of every stretch). `eyes_review/ingest_states.py` loads them into the review database.